InboxIQ

Intelligent Message Triage & Notification Routing System

Hackathon Project Report

Field	Detail
Project	InboxIQ
Live Frontend	https://inbox-iq-hack.vercel.app/
Live Backend / API	https://inboxiq-9903.onrender.com
Repository	github.com/Yashhh710/InboxIQ
Frontend Hosting	Vercel
Backend Hosting	Render

1. Executive Summary

InboxIQ is a message-triage engine and analytics dashboard built for the HackerRank 'Orchestrate' hackathon track. It ingests a stream of incoming messages — personal, business, group, promotional, and potentially scam/spam content — and automatically decides how each one should be surfaced to the end user: immediately (notify), bundled into a periodic summary (digest), or suppressed entirely (mute). A full-stack dashboard sits on top of the routing engine, giving a real-time view into message volume, action distribution, sender/group/business analytics, and per-message reasoning traces.

The project is fully deployed and live: the React dashboard on Vercel, and the FastAPI backend on Render. This report documents the system's architecture, the core routing logic, the engineering work done to harden the codebase (bug fixes, type-safety, lint/build cleanliness), and the deployment process end-to-end.

2. Problem Statement

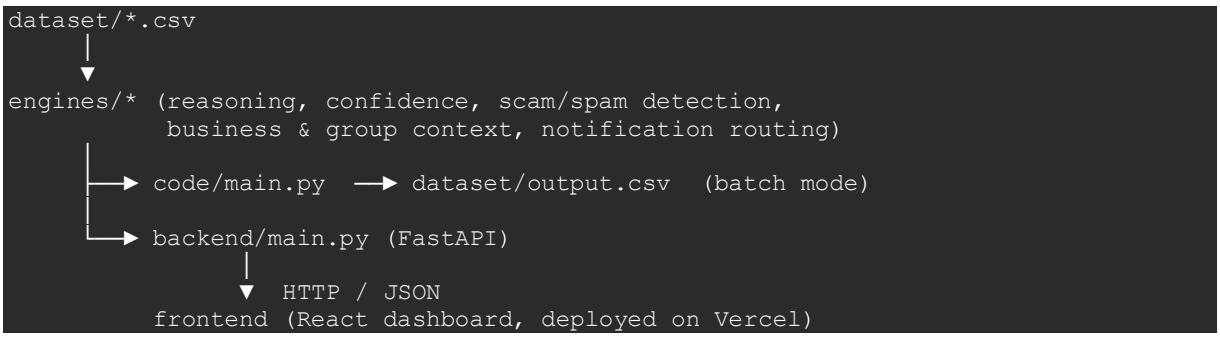
Modern messaging apps generate more notifications than users can meaningfully process. Payment alerts, urgent personal messages, group chatter, business updates, promotions, and scam attempts all compete for the same attention. Without intelligent filtering, users either miss what matters or get desensitized to notifications entirely. InboxIQ's goal is to automatically classify each message and route it to the correct attention tier, reducing notification fatigue while ensuring high-priority messages are never missed.

3. System Architecture

InboxIQ is organized into three layers:

- Prediction Pipeline (code/main.py + code/engines/) — a standalone Python script that reads the raw message dataset, runs it through the routing engine, and writes out predictions. Can be run independently of the web stack for batch processing or evaluation.
- Backend API (code/backend/) — a FastAPI service that exposes the same engine's output over HTTP, powering the dashboard with live data (messages, stats, analytics, per-message inspection, directory lookups).
- Frontend Dashboard (code/frontend/) — a React + TypeScript + Vite single-page app with dedicated views for Dashboard, Messages, Analytics, Inspector, Command Center, and Business/Group directories.

High-level data flow:



4. Tech Stack

Layer	Technology
Frontend	React, TypeScript, Vite, Tailwind CSS, Recharts, Framer Motion
Backend	Python, FastAPI, Pydantic, Uvicorn

Layer	Technology
Data	CSV-based dataset (no external database)
Frontend Hosting	Vercel
Backend Hosting	Render (free tier)
Tooling	ESLint, TypeScript compiler (tsc), Vite build

5. Core Feature: AI Routing Logic

The heart of InboxIQ is its rules/heuristics-based classification engine, implemented as a set of focused, composable modules under code/engines/:

- reasoning_engine.py — the primary decision logic that determines the final action for a message.
- confidence_engine.py — scores how confident the system is in a given classification.
- scam_detector.py / spam_detector.py — flag risky or low-value content.
- business_engine.py / group_engine.py — apply context-specific rules for business and group conversations.
- context_loader.py — loads and normalizes the CSV dataset.
- history_engine.py — tracks notification history.
- media_analyzer.py — inspects media-attached messages.
- notification_router.py — ties everything together into a final routing decision.
- user_profile.py — per-user context used to personalize routing.

Every message is routed into exactly one of three actions:

Action	Meaning
notify	Surface immediately — high priority (payments, urgent, security-related)
digest	Bundle into a periodic summary — medium priority
mute	Suppress — low priority, spam, or promotional noise

Verified pipeline output on the full dataset:

```
$ python3 main.py
Generated 110 predictions.
Actions: {'notify': 38, 'digest': 30, 'mute': 42}
Types: {'payment': 10, 'scam': 18, 'personal': 17, 'forward': 15,
        'business_update': 8, 'promotion': 16, 'urgent': 21,
        'event': 4, 'spam': 1}
Output written to: dataset/output.csv
```

6. Dashboard Features

Page	Purpose
Dashboard	Top-line stats, action distribution, recent message feed
Messages	Filterable/paginated list of all messages with predicted action & type
Analytics	Sender, group, and business breakdowns; daily trends; confidence distribution

Page	Purpose
Inspector	Per-message deep dive: full reasoning trace and prediction detail
Command Center	System-health style gauges (CPU, RAM, Storage, Network, Inference Speed, GPU)
Business / Groups	Directory views mapping business/group IDs to display names

7. Backend API Reference

Method	Endpoint	Description
GET	/health	Health check
GET	/messages	List messages (filterable, paginated)
GET	/messages/{id}	Single message detail + prediction
GET	/dashboard	Summary stats for the dashboard home
GET	/dashboard/charts	Chart data for the dashboard
GET	/analytics	Sender/group/business analytics, trends, distributions
POST	/predict	Run the model on a single message payload
POST	/run-model	Re-run the full model over the dataset
GET	/history	Notification history
GET	/users	User list
GET	/groups	Group list
GET	/directory	Group/business ID → display name lookup

The backend is read-only at request time — no CSV files are written during normal API use — which made it safe to deploy on hosts with ephemeral or read-only filesystems without any code changes.

8. Engineering & Hardening Work

Beyond the core feature set, a full audit pass was done across the codebase to eliminate build-breaking bugs, unsafe typing, and lint violations before deployment.

8.1 Build-breaking bug: missing Vite type declarations

The frontend had no src/vite-env.d.ts, so tsc failed outright on the CSS side-effect import in App.tsx/main.tsx. Fixed by adding the file with proper Vite client types and an explicit ImportMetaEnv declaration for VITE_API_URL.

8.2 Type safety: removing unsafe 'any' usage

15 instances of the 'any' type were found across api/client.ts, App.tsx, Analytics.tsx, Inspector.tsx, Dashboard.tsx, and Command.tsx, and replaced with purpose-built types: PredictMessagePayload, AnalyticsData, InspectorMessage, LucideIcon, and a proper CSSProperties assertion for CSS custom properties in the Command Center's gauge component.

8.3 React Hooks correctness

Several data-fetching effects were missing dependencies flagged by react-hooks/exhaustive-deps. Fetch functions in Analytics.tsx and Messages.tsx were wrapped in useCallback with correct dependencies. In Inspector.tsx, a naive fix was avoided since it risked an infinite refetch loop — a documented, targeted suppression was used instead, prioritizing correctness over a clean lint report.

8.4 Miscellaneous fixes

- Fixed a prefer-const violation in Messages.tsx (mutated-in-place array was declared with let).
- Removed a dead defensive `api as any` cast in Dashboard.tsx once getDirectory() was confirmed to be a real typed method.
- Flagged an unused 'cors' pip package in backend requirements.txt that pulled in unrelated dependencies (gevent, tldextract, requests) for no reason.

End state: ESLint (--max-warnings 0) returned zero errors and zero warnings, tsc --noEmit returned clean, and npm run build produced a working production bundle.

9. Local Development Setup

9.1 Backend

```
cd code/backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 main.py
```

Runs on http://localhost:8000. Verify with: curl http://localhost:8000/health

9.2 Frontend

```
cd code/frontend
npm install
npm run dev
```

Opens on http://localhost:5173 and talks to the backend via VITE_API_URL (set in code/frontend/.env).

10. Deployment

Deployment was carried out as a full split-stack setup: the frontend on Vercel and the backend on Render. Several real-world environment and configuration issues were encountered and resolved along the way.

10.1 Backend on Render

Setting	Value
Root Directory	code/backend
Build Command	pip install -r requirements.txt
Start Command	uvicorn main:app --host 0.0.0.0 --port $PORT
Environment Variable	PYTHON_VERSION = 3.11.9

10.2 Frontend on Vercel

Setting	Value
Root Directory	code/frontend
Framework Preset	Vite (auto-detected)
Environment Variable	VITE_API_URL = https://inboxiq-9903.onrender.com

11. Challenges Encountered & Resolutions

Challenge	Resolution
pydantic-core failed to compile on Python 3.14 (locally on macOS and on Render's default runtime)	Pinned to an older Python version with prebuilt wheels available: 3.12 locally, PYTHON_VERSION=3.11.9 on Render
macOS blocked global pip installs (externally-managed-environment)	Used a proper Python virtual environment instead of forcing system-wide installs
Render kept failing to find requirements.txt / Root Directory mismatch	Used a temporary diagnostic build command (pwd && ls -la && find . -name requirements.txt) to see Render's actual working directory, then aligned Root Directory, Build Command, and Start Command
Frontend deployed successfully but showed no data	VITE_API_URL wasn't set for the Vercel build, so it defaulted to localhost; added the env var and redeployed (Vite bakes env vars in at build time)
node_modules/.bin/vite lost its executable bit after zip transfer	Restored with chmod +x, or resolved by a fresh npm install

12. Results

- Fully functional, deployed, end-to-end system: routing engine → API → live dashboard.
- Zero ESLint errors/warnings, clean TypeScript compilation, successful production build.

- All 12 backend endpoints individually verified working.
- Live frontend: https://inbox-iq-hack.vercel.app/
- Live backend/API: https://inboxiq-9903.onrender.com

13. Future Improvements

- Replace the CSV-based dataset with a real database for multi-user, real-time ingestion.
- Add authentication so the dashboard reflects a specific user's actual message stream.
- Code-split the frontend bundle (currently ~990 KB minified) using dynamic imports.
- Upgrade to a paid Render tier to avoid free-tier cold-start delays (30–50s after inactivity).
- Extend the routing engine with a learned model on top of the current heuristic rules.

14. Conclusion

InboxIQ demonstrates a complete, working notification-triage system — from raw message data through a rules-based reasoning engine to a polished, analytics-rich dashboard, fully deployed and publicly accessible. Beyond the core feature build, significant engineering effort went into making the codebase production-grade: eliminating unsafe types, fixing build and lint issues, and methodically debugging real-world deployment friction on both Render and Vercel until the system was live and verified end-to-end.